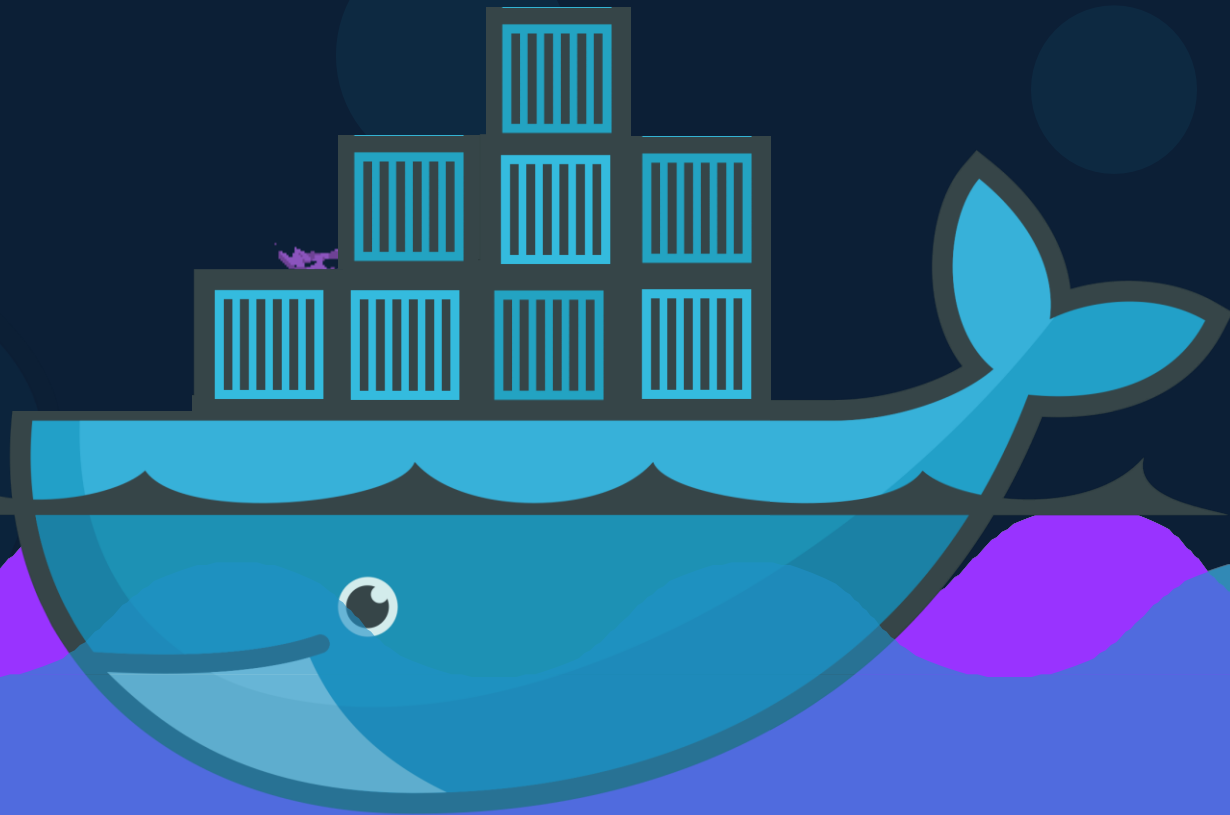


docker

NOURAN
ZOUABI



NZ

Objectives



- Notion de base docker
- Docker File
- Docker Compose
- Docker swarm vs Kubernetes

docker

Notions de base

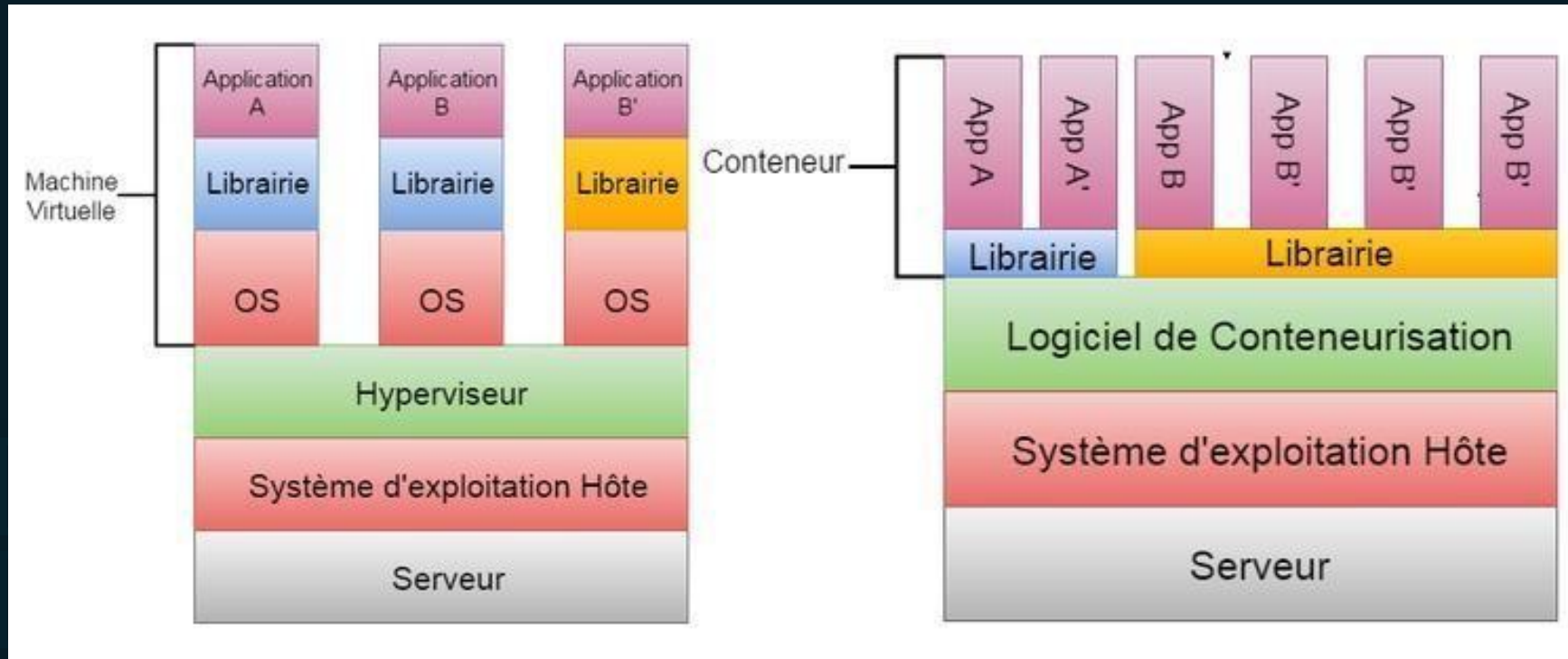
Virtualisation vs cloud

- **La virtualisation:** La virtualisation est une technologie permettant de créer et d'exécuter une ou plusieurs représentations virtuelles d'un ordinateur et de ses différentes ressources sur une même machine physique.
- La virtualisation a eu le succès grâce au **cloud computing**: C'est un Data center ou une infrastructure offerte par un fournisseur dans laquelle la puissance de calcul et le stockage sont gérés par des serveurs distants auxquels les usagers se connectent via une liaison Internet sécurisée.

- Elasticité rapide
- Modèle Pay as You Go



Machine virtuelle vs Conteneurs



- On distingue deux types de virtualisation tels que:
 - ✓ La virtualisation lourde (VM)
 - ✓ La virtualisation légère (Conteneurs)

- **Avantages des VMs**

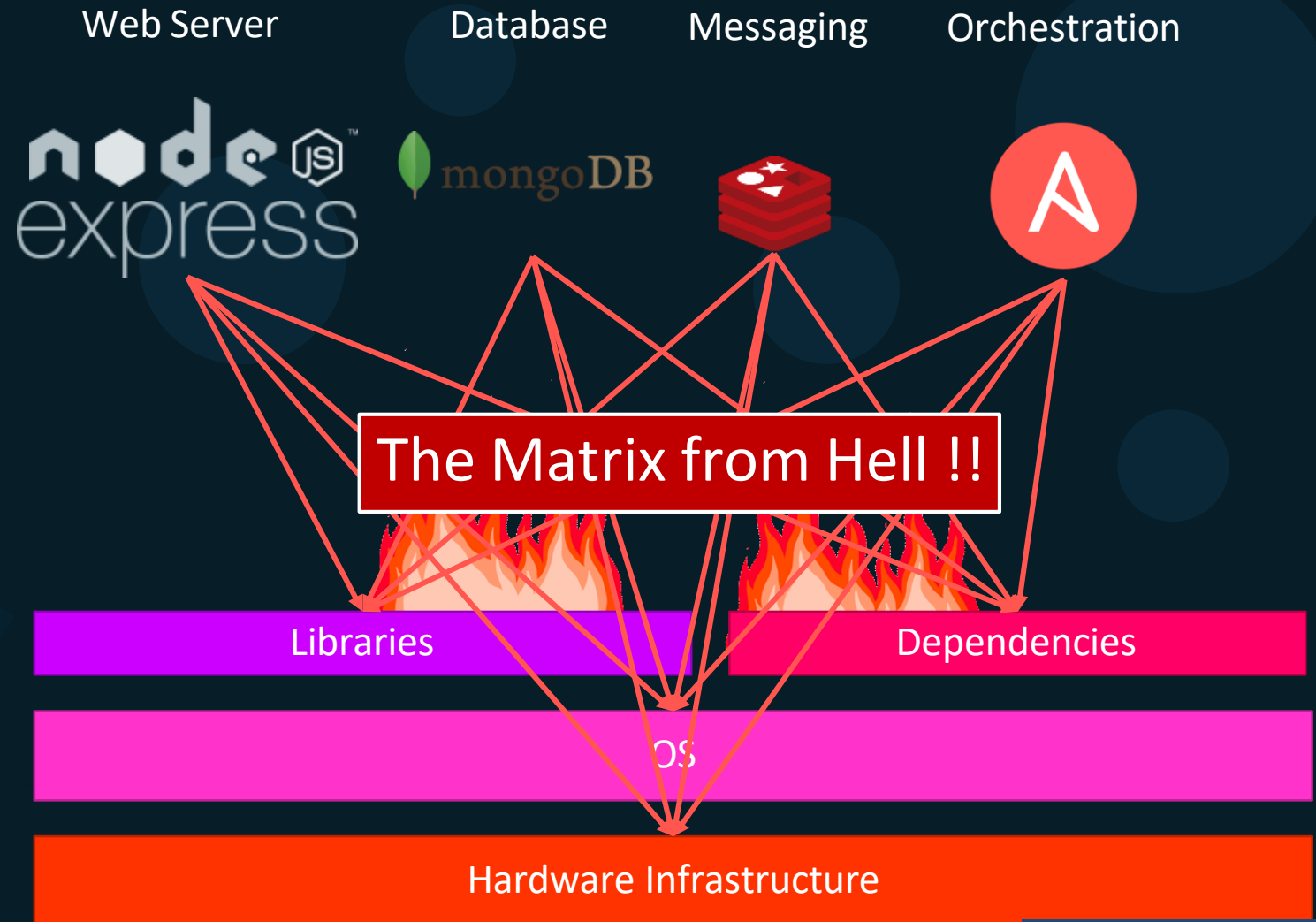
- ✓ Une meilleure exploitation des ressources
- ✓ Une machine physique est divisée en plusieurs machines virtuelles

- **Limites des VMs**

- ✓ Chaque VM a besoin des ressources (CPU, Stockage (Disque), RAM, OS invité)
- ✓ En augmentant les VMs, on demande plus de ressources
 - Chaque OS invité alloue ses propres ressources
 - Gaspillage
 - Portabilité d'applications n'est pas garantie

Pourquoi on utilise Docker ?

- Compatibility/Dependency
- Long setup time
- Different Dev/Test/Prod environments



Conteneurs

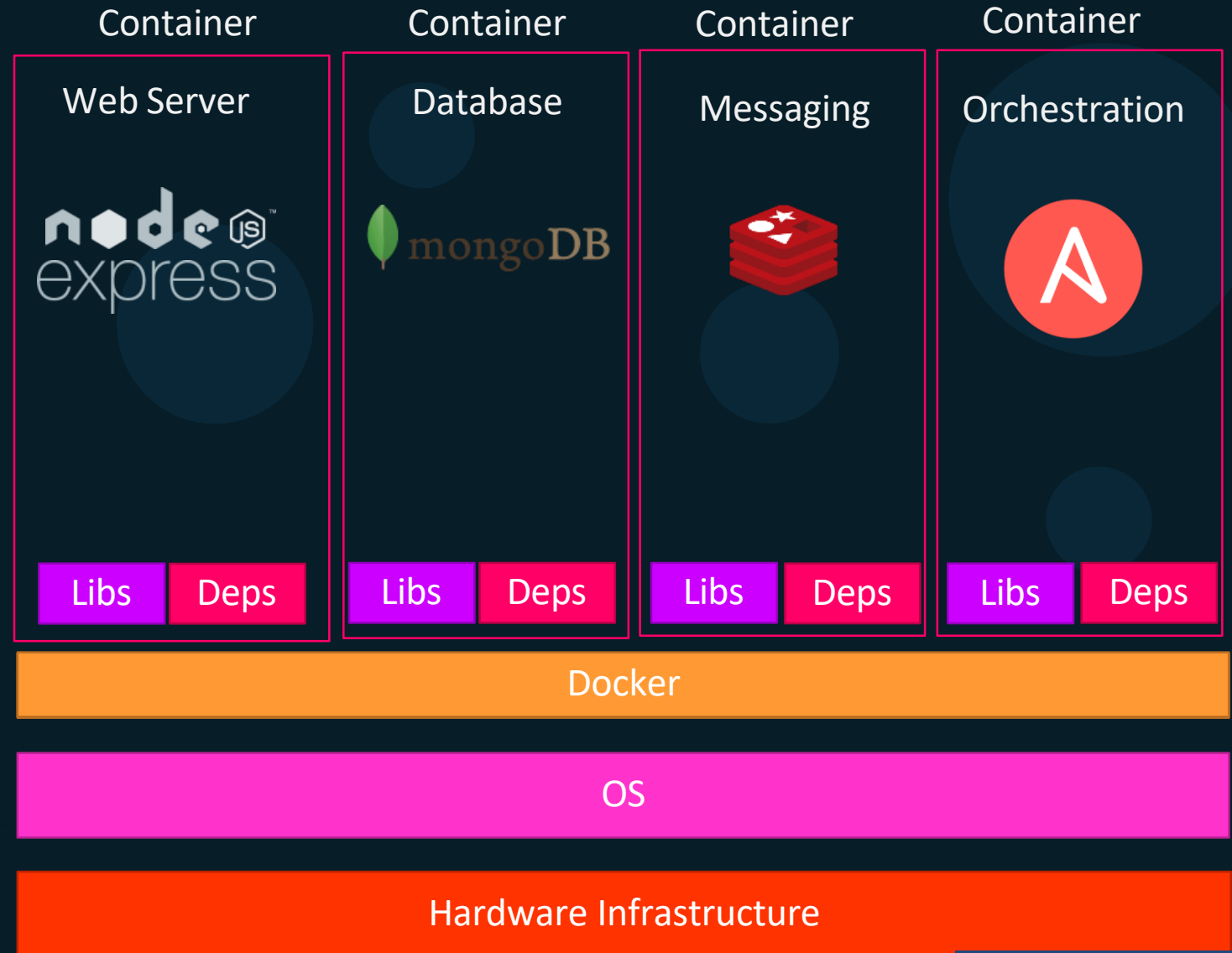
La virtualisation légère ou à base des conteneurs:

- Offre tous les services, scripts, API, librairies dont une application a besoin favorisant ainsi une juste utilisation des ressources.
- Repose sur la création de conteneurs isolés les uns des autres sur un **OS Commun**.
- Les conteneurs indépendants partagent un **OS commun** et un même **espace mémoire**.

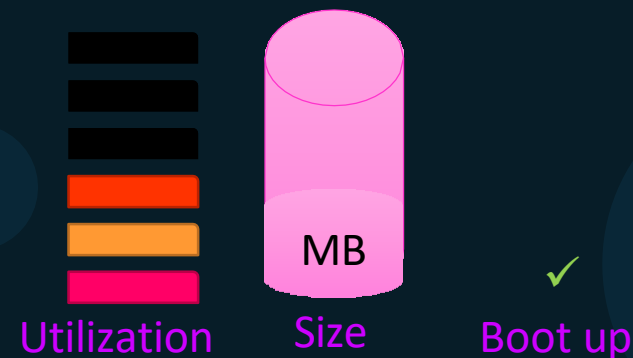
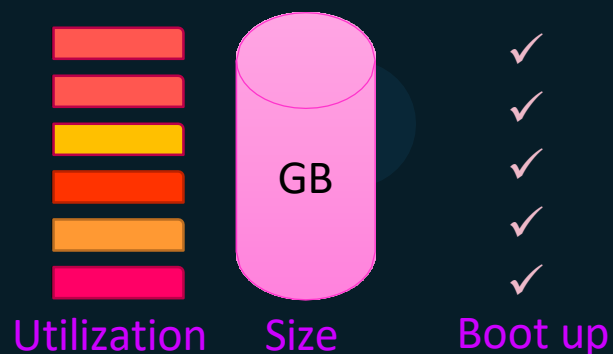
- Un conteneur est une enveloppe (emballage) contenant toutes les ressources nécessaires pour faire fonctionner une application donnée (Environnement d'exécution comme JDK, livrable de l'application, dépendances nécessaires)

Conteneurs

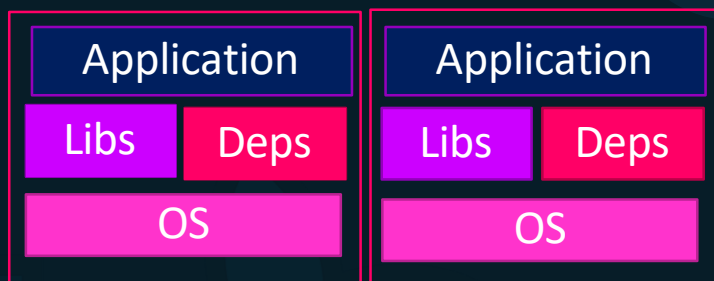
- Containerize Applications
- Run each service with its own dependencies in separate containers



Machine virtuelle vs Conteneurs



Virtual Machine Virtual Machine



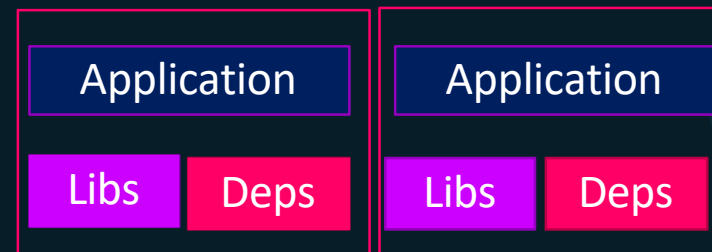
Hypervisor

OS

Hardware Infrastructure

Container

Container



Docker

OS

Hardware Infrastructure

Machine virtuelle vs Conteneurs

VM	Container
Une virtualisation lourde	Une virtualisation légère
À base de VM	À base des Conteneurs
Virtualisation des ressources hardware (CPU, RAM, disque, ...)	Virtualisation au niveau du système d'exploitation (de l'OS)
Machine invité (machine virtuelle)	Conteneur
Image ISO (OS Complet)	Librairies nécessaires
Hyperviseur sur la Machine hôte	Moteur de conteneur sur la machine hôte
Démarrage plus lent	Démarrage en quelques secondes
Entièrement isolé et donc plus sécurisé	Isolation au niveau du processus et donc moins sécurisée

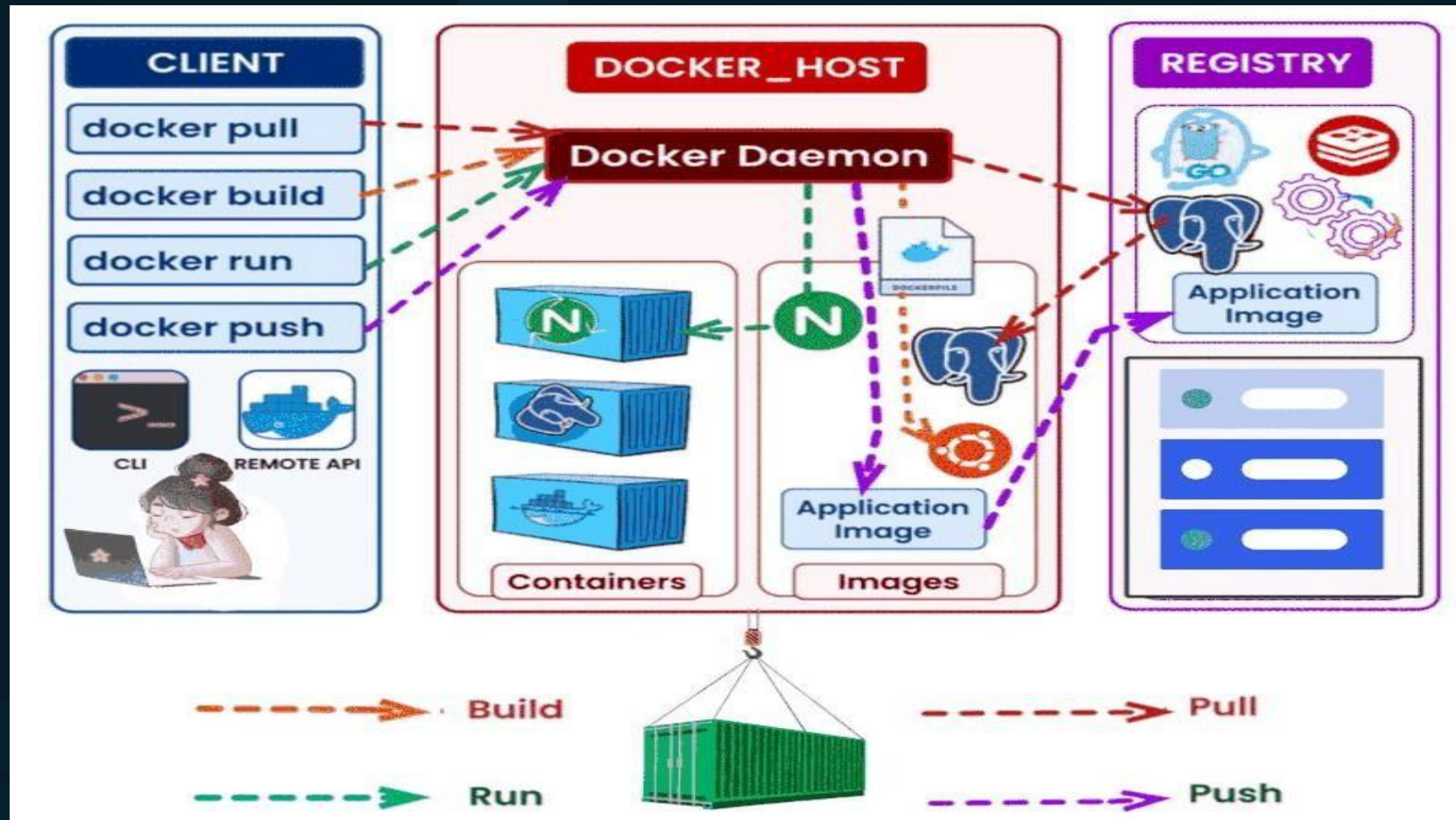
Docker - Définition

- Docker permet d'embarquer une application dans un ou plusieurs containers logiciels qui pourra s'exécuter sur n'importe quel serveur machine, qu'il soit physique ou virtuel.
- On distingue deux versions de docker:
 - Docker **Enterprise Edition** : Docker EE payante
 - Docker **Community Edition** : Docker CE gratuite



docker

Architecture



Docker-Components

Images : Ils sont des modèles en lecture seule avec des instructions pour créer un conteneur Docker. L'image Docker peut être extraite d'un hub Docker et utilisée telle quelle, ou vous pouvez ajouter des instructions supplémentaires (via un Docker File) à l'image de base et créer une image Docker nouvelle et modifiée.

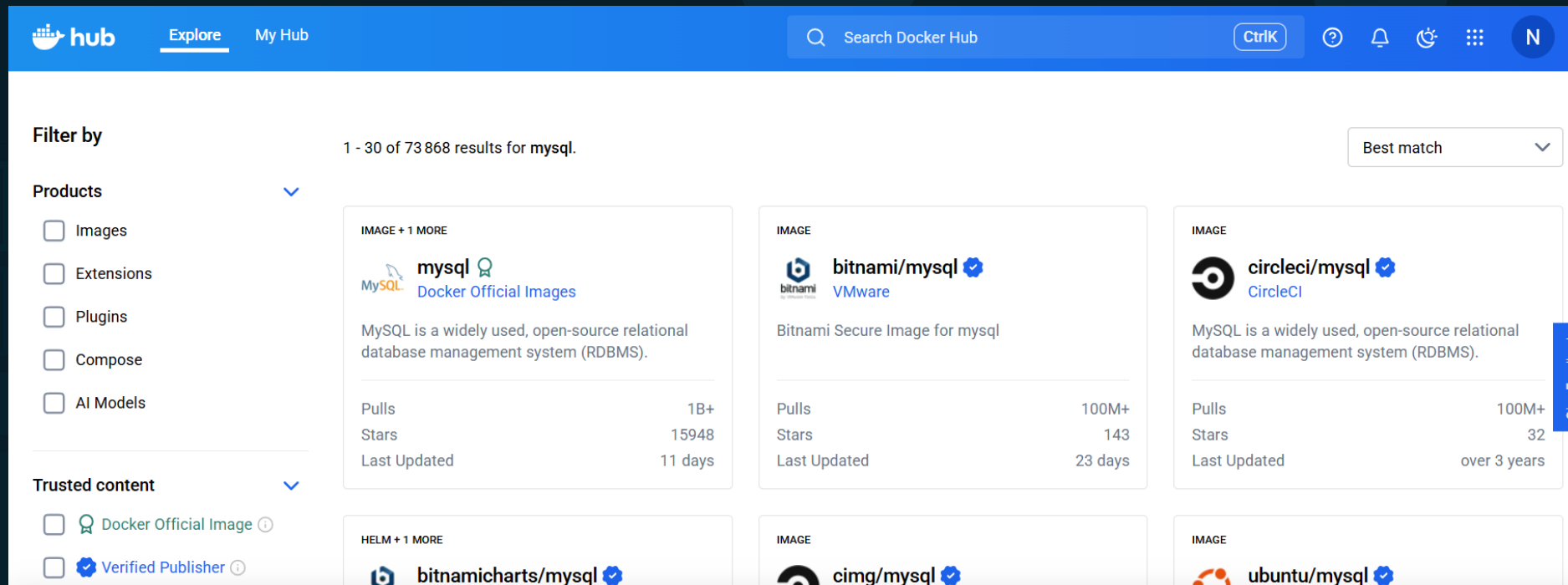
Conteneurs : Une fois que vous avez exécuté une image Docker, elle crée un conteneur Docker. Toutes les applications et leur environnement s'exécutent à l'intérieur de ce conteneur. Vous pouvez utiliser une interface (Docker Desktop, ...) ou la CLI Docker pour démarrer, arrêter, supprimer un conteneur Docker.

NZ

Le conteneur est une instance d'une image exécutée

Docker-Composants

Registres Docker: C'est l'emplacement où les images Docker sont stockées. Il s'agit d'un registre docker public (Docker Hub) ou d'un registre docker privé.



Docker-Image

- Pour créer une image, nous pouvons utiliser les solutions suivantes :
 - DockerHub (importer une image depuis le cloud Docker)
 - Utiliser un fichier Dockerfile pour créer une image à travers les commandes
 - A partir d'un registry privé (en entreprise)
 - A partir d'un conteneur (Image => conteneur => image)

DockerFile

- Les instructions de base que peut contenir un DockerFile sont les suivantes :

- **FROM** permet de définir depuis quelle base votre image va être créée
- **LABEL** permet de définir l'auteur de l'image
- **RUN** permet de lancer une commande
- **ADD** permet de copier un fichier depuis la machine hôte ou depuis une URL
- **EXPOSE** permet d'exposer un port du container vers l'extérieur
- **CMD** détermine la commande qui sera exécutée lorsque le container démarrera
- **ENTRYPOINT** permet d'ajouter une commande qui sera exécutée par défaut
- **WORKDIR** permet de définir le dossier de travail pour toutes les autres commandes
- **ENV** permet de définir des variables d'environnements
- **VOLUMES** permet de créer un point de montage qui permettra de persister les données

Dockerfile



Public Docker registry - dockerhub

Dockerfile

INSTRUCTION

ARGUMENT

Dockerfile

FROM Ubuntu

RUN apt-get update

RUN apt-get install python

RUN pip install flask

RUN pip install flask-mysql

COPY . /opt/source-code

ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run

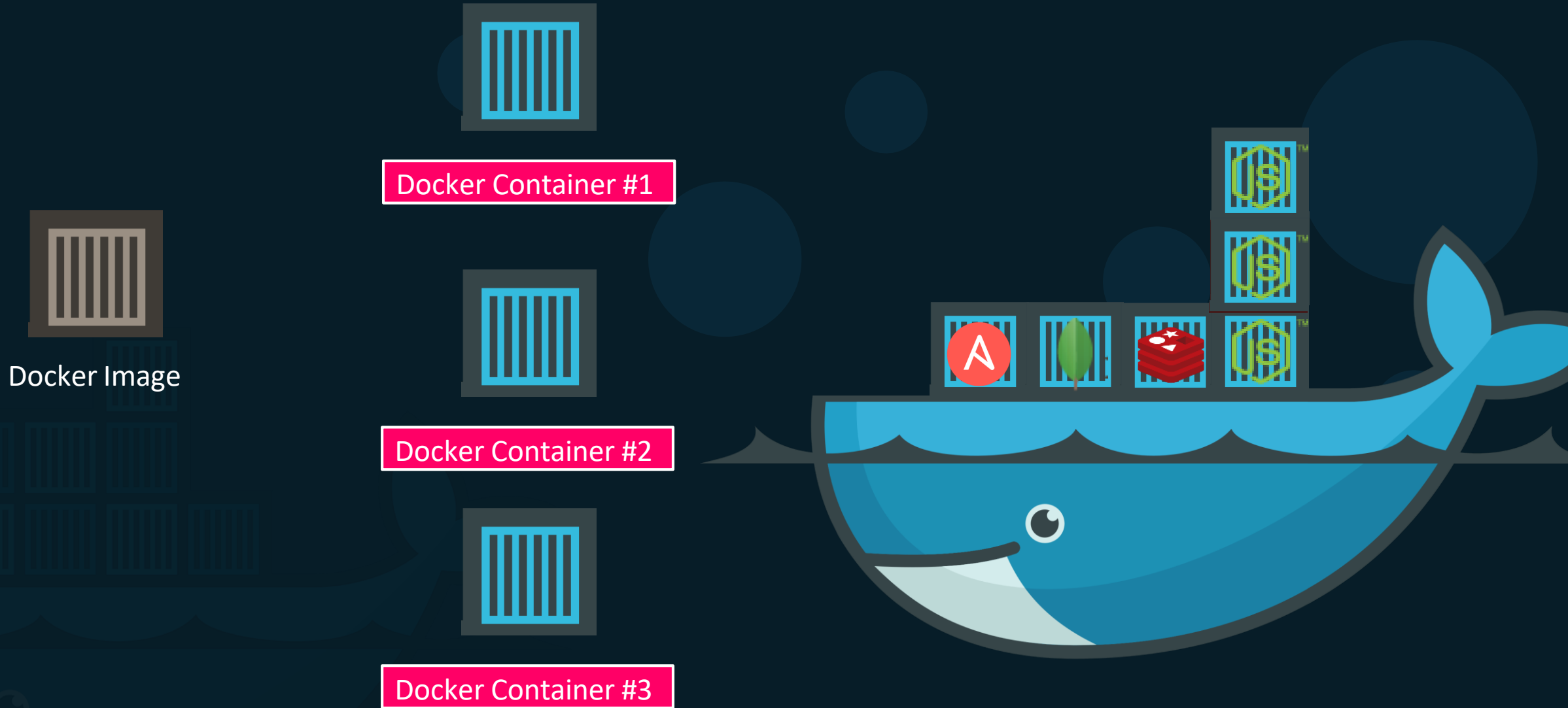
Start from a base OS or another image

Install all dependencies

Copy source code

Specify Entrypoint

How is it done?



docker

commands

▶ docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
796856ac413d	nginx	"nginx -g 'daemon of...'"	7 seconds ago	Up 6 seconds	80/tcp	silly_sammet

▶ docker ps -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
796856ac413d	nginx	"nginx -g 'daemon of...'"	7 seconds ago	Up 6 seconds	silly_sammet
cff8ac918a2f	redis	"docker-entrypoint.s..."	6 seconds ago	Exited (0) 3 seconds ago	relaxed_aryabhata

▶ docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	f68d6e55e065	4 days ago	109MB
redis	latest	4760dc956b2d	15 months ago	107MB
ubuntu	latest	f975c5035748	16 months ago	112MB
alpine	latest	3fd9065eaf02	18 months ago	4.14MB

▶ docker stop silly_sammet

silly_sammet

▶ docker rmi nginx

Untagged: nginx:latest
Untagged: nginx@sha256:96fb261b66270b900ea5a2c17a26abbfab95506e73c3a3c65869a6dbe83223a
Deleted: sha256:f68d6e55e06520f152403e6d96d0de5c9790a89b4cfc99f4626f68146fa1dbdc
Deleted: sha256:1b0c768769e2bb66e74a205317ba531473781a78b77feef8ea6fd7be7f4044e1
Deleted: sha256:34138fb60020a180e512485fb96fd42e286fb0d86cf1fa2506b11ff6b945b03f
Deleted: sha256:cf5b3c6798f77b1f78bf4e297b27cfa5b6caa982f04caeb5de7d13c255fd7a1e

▶ docker run nginx

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
fc7181108d40: Already exists
d2e987ca2267: Pull complete
0b760b431b11: Pull complete
Digest:
sha256:96fb261b66270b900ea5a2c17a26abbfabe95506e73c3a3c65869a6dbe83223a
Status: Downloaded newer image for nginx:latest
```

▶ docker pull nginx

```
Using default tag: latest
latest: Pulling from library/nginx
fc7181108d40: Pull complete
d2e987ca2267: Pull complete
0b760b431b11: Pull complete
Digest:
sha256:96fb261b66270b900ea5a2c17a26abbfabe95506e73c3a3c65869a6dbe83223a
Status: Downloaded newer image for nginx:latest
```

Run – attach and detach

```
▶ docker run kodecloud/simple-webapp
```

```
This is a sample web application that displays a colored background.  
* Serving Flask app "app" (lazy loading)  
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

```
▶ docker run -d kodecloud/simple-webapp
```

```
a043d40f85fef414254e4775f9336ea59e19e5cf597af5c554e0a35a1631118
```

Run – PORT mapping

```
docker run kodecloud/webapp
```

```
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```

<http://172.17.0.2:5000>

Internal IP

```
docker run -p 80:5000 kodecloud/simple-webapp
```

```
docker run -p 8000:5000 kodecloud/simple-webapp
```

```
docker run -p 8001:5000 kodecloud/simple-webapp
```

```
docker run -p 3306:3306 mysql
```

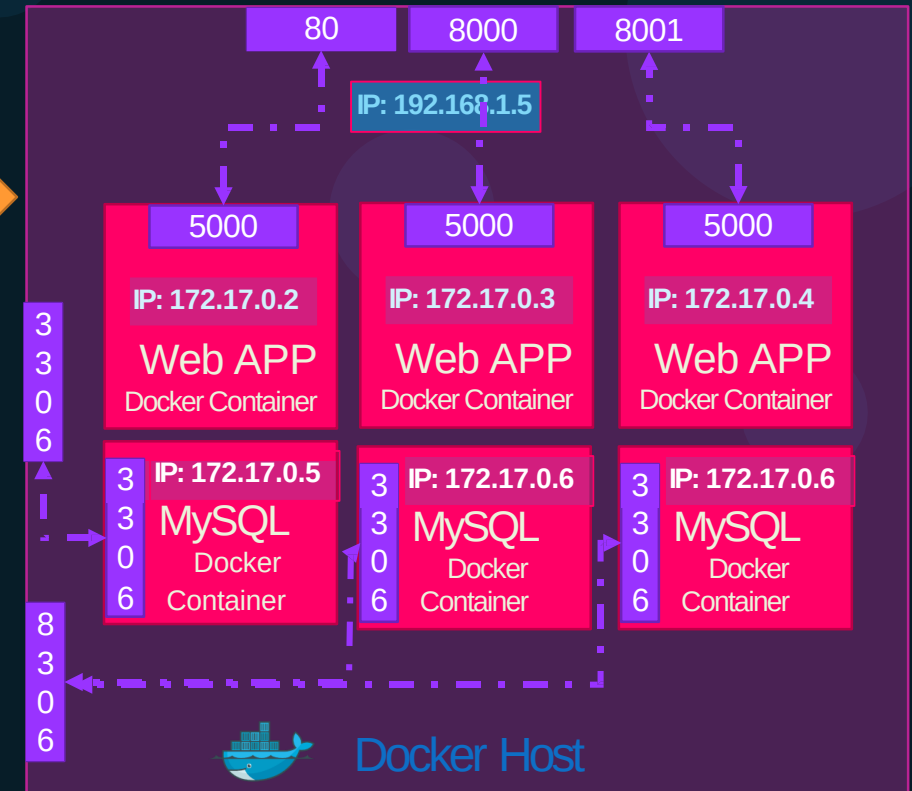
```
docker run -p 8306:3306 mysql
```

```
docker run -p 8306:3306 mysql
```

```
root@osboxes:/root # docker run -p 8306:3306 -e MYSQL_ROOT_PASSWORD=pass mysql
docker: Error response from daemon: driver failed programming external connectivity on endpoint boring_bhabha (5079d342b7e8ee11c71d46): Bind for 0.0.0.0:8306 failed: port is already allocated.
```



<http://192.168.1.5:80>

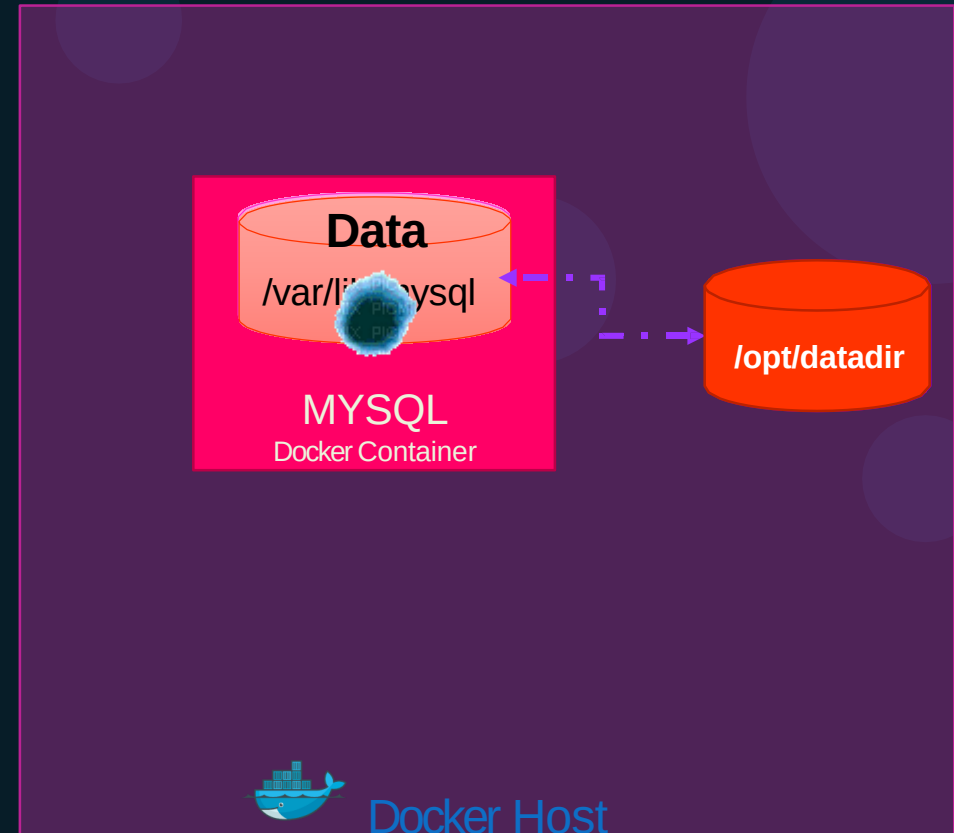


RUN – Volume mapping

```
docker run mysql
```

```
docker stop mysql  
docker rm mysql
```

```
docker run -v /opt/datadir:/var/lib/mysql mysql
```





docker compose

NZ

Docker compose

```
docker run mmumshad/simple-webapp
```

```
docker run mongodb
```

```
docker run redis:alpine
```

```
docker run ansible
```

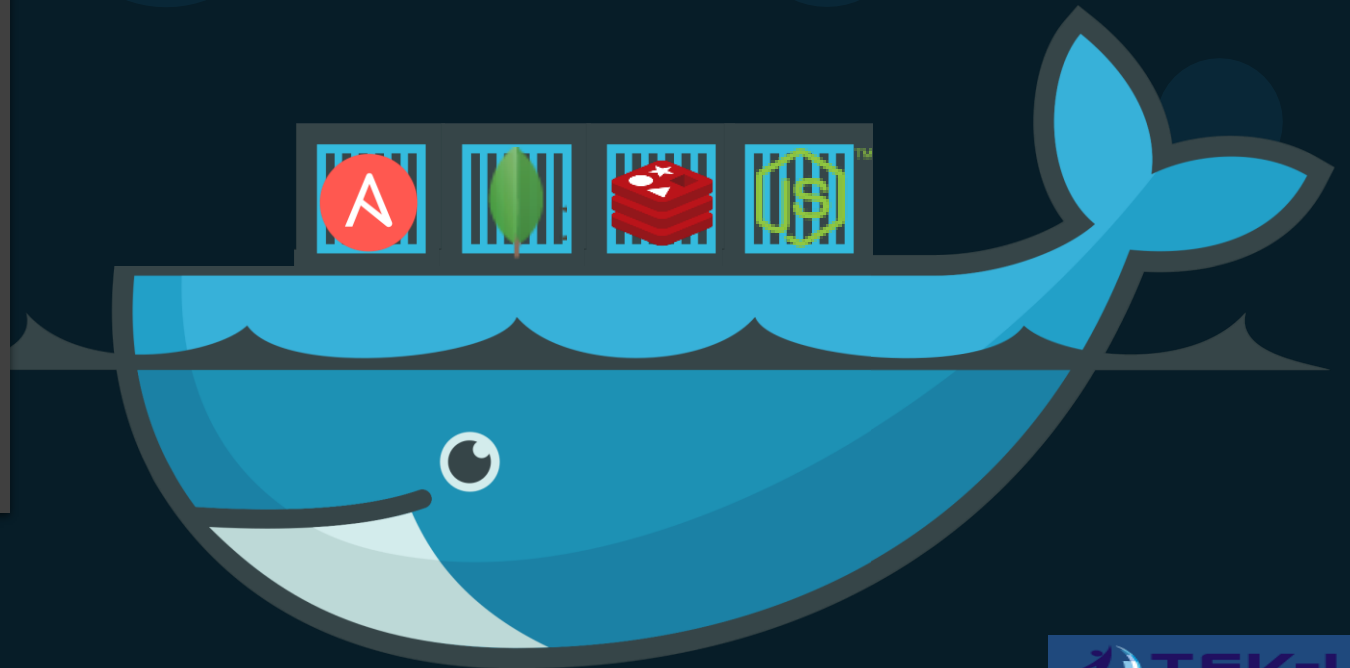
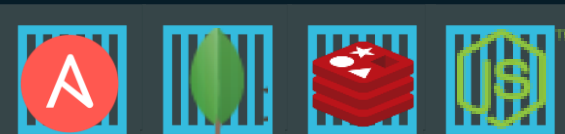
```
docker-compose.yml
```

```
services:
  web:
    image: "mmumshad/simple-webapp"
  database:
    image: "mongodb"
  messaging:
    image: "redis:alpine"
  orchestration:
    image: "ansible"
```

```
docker-compose up
```



Public Docker registry - dockerhub



• Docker Swarm VS Kubernetes •



Docker Swarm



kubernetes



Merci pour votre attention